

Spark Exercise

Apache Spark is an excellent tool for data engineering projects due to its robust ability to process large-scale data efficiently through distributed computing. Spark's in-memory processing capabilities significantly enhance the speed of data operations, making it ideal for handling big data workloads. It supports various data sources and formats, offering versatility in data ingestion and transformation. Additionally, Spark's rich API supports multiple programming languages such as Python, Java, and Scala, catering to diverse developer preferences. Its ecosystem, which includes libraries for SQL, machine learning, and graph processing, provides a comprehensive suite for building complex data pipelines and analytics, making it a powerful and flexible choice for data engineering tasks.

Spark RDD

Spark RDD (Resilient Distributed Dataset) is a fundamental data structure in Apache Spark that enables fault-tolerant, distributed processing of large datasets across multiple nodes in a cluster. Spark RDDs provide a higher-level abstraction for performing distributed data processing tasks, including both map (transformations) and reduce (aggregations) operations.

Import Necessary Libraries

```
In [1]: import os
import pandas as pd
import numpy as np
from pyspark.sql import SparkSession
from pyspark import SparkContext, SparkConf
```

Spark Context and Session

Initialize Spark Context and Spark Session

```
In [2]: # Spark Konfiguration für den Cluster erstellen
conf = SparkConf() \
    .setAppName("BDENG_Spark_Exercise") \
    .setMaster("spark://172.29.16.102:7077")

# SparkSession und SparkContext initialisieren
spark = SparkSession.builder.config(conf=conf).getOrCreate()
sc = spark.sparkContext

print("Spark Version:", spark.version)
print("Verbindung zum Cluster erfolgreich hergestellt!")
```

```
Setting default log level to "WARN".
To adjust logging level use sc.setLogLevel(newLevel). For SparkR, use setLogLevel(newLevel).
26/07/01 15:09:53 WARN NativeCodeLoader: Unable to load native-hadoop library for your platform... using builtin-java classes where applicable
26/07/01 15:09:53 WARN Utils: Service 'SparkUI' could not bind on port 4040. Attempting port 4041.
26/07/01 15:09:53 WARN Utils: Service 'SparkUI' could not bind on port 4041. Attempting port 4042.
Spark Version: 3.5.6
Verbindung zum Cluster erfolgreich hergestellt!
```

Load Data into RDD

```
In [3]: import pandas as pd

# 1. Datei mit Pandas einlesen
pdf_local = pd.read_csv('cleaned_beer_data.csv')

# 2. Die Spaltennamen bereinigen
pdf_local.columns = pdf_local.columns.str.strip()

# 3. Wir simulieren die Roh-CSV-Zeilen ohne Header für das RDD
csv_lines = pdf_local.to_csv(index=False, header=False).strip().split('\n')
data_rdd = sc.parallelize(csv_lines)

print(f"Anzahl geladener Zeilen im RDD: {data_rdd.count()}")
```

```
[Stage 0:> (0 + 2) / 2]
Anzahl geladener Zeilen im RDD: 32
```

Map Operation

Split data into individual parts and create key-value pairs

```
In [4]: def parse_line(line):
        parts = line.split(',')
        year = int(float(parts[0]))
        consumption = float(parts[10])
        return (year, consumption)

        # Sicherer gegen Formatierungsfehler
        # Verbrauch_pro_Kopf_L ist an Index 10

mapped_rdd = data_rdd.map(parse_line)
```

Reduce Operation

Reduce your key-value pairs

```
In [5]: # Jahr in Jahrzehnt umrechnen und addieren
reduced_rdd = mapped_rdd.map(lambda x: (int(x[0] / 10) * 10, x[1])).reduceByKey()
```

Collect Results

Because of lazy evaluation, the map-reduce operation is performed only now. Show what you calculated.

```
In [6]: # Berechnung triggern
results = reduced_rdd.sortByKey().collect()

print("Berechnete Ergebnisse:")
for decade, total_cons in results:
    print(f"{decade}er Jahre: {total_cons:.2f} Liter")
```

```
Berechnete Ergebnisse:
1990er Jahre: 763.90 Liter
2000er Jahre: 1126.00 Liter
2010er Jahre: 978.30 Liter
2020er Jahre: 499.40 Liter
```

Save Results

```
In [7]: import os
import shutil

# Alten Ordner Löschen falls vorhanden
if os.path.exists("rdd_output_decades"):
    shutil.rmtree("rdd_output_decades")

# Ergebnisse in eine normale sofort sichtbare Textdatei schreiben
with open("rdd_ergebnis_lesbar.txt", "w") as f:
    for decade, total_cons in results:
        f.write(f"{decade}er Jahre: {total_cons:.2f} Liter\n")

print("Erfolgreich! Die Datei 'rdd_ergebnis_lesbar.txt' wurde erstellt und ist l
```

Erfolgreich! Die Datei 'rdd_ergebnis_lesbar.txt' wurde erstellt und ist links im Menü sichtbar!

Spark DataFrame

Spark DataFrame is a distributed collection of data organized into named columns, designed for efficient data manipulation and analysis in Apache Spark. It is used for various data processing tasks such as data ingestion, transformation, querying, and analysis in Apache Spark, providing a high-level abstraction that simplifies working with structured data.

Load Data into DataFrame

```
In [8]: # Den Pandas DataFrame direkt in einen Spark DataFrame konvertieren
df = spark.createDataFrame(pdf_local)
```

```
26/07/01 15:09:59 WARN FileSystem: Cannot load filesystem: java.util.ServiceConfigurationError: org.apache.hadoop.fs.FileSystem: com.google.cloud.hadoop.fs.gcs.GoogleHadoopFileSystem Unable to get public no-arg constructor
26/07/01 15:09:59 WARN FileSystem: java.lang.NoClassDefFoundError: com/google/api/client/http/HttpRequestInitializer
26/07/01 15:09:59 WARN FileSystem: java.lang.ClassNotFoundException: com.google.api.client.http.HttpRequestInitializer
```

View DataFrame Schema

```
In [9]: df.printSchema()
```

```
root
|-- Jahr: long (nullable = true)
|-- Absatz_Bier_hl: double (nullable = true)
|-- Absatz_Biermisch_hl: double (nullable = true)
|-- Versteuert_hl: double (nullable = true)
|-- Steuerfrei_hl: double (nullable = true)
|-- Export_EU_hl: double (nullable = true)
|-- Export_Drittland_hl: double (nullable = true)
|-- Haustrunk_hl: double (nullable = true)
|-- Braustaetten_Anzahl: double (nullable = true)
|-- Verbrauch_Bier_hl: double (nullable = true)
|-- Verbrauch_pro_Kopf_l: double (nullable = true)
```

View DataFrame Data

```
In [10]: # Zeigt nur die ersten 5 Zeilen
df.show(5)
```

```

+----+-----+-----+-----+-----+-----+
+-----+-----+-----+-----+-----+-----+
-----+
|Jahr|Absatz_Bier_hl|Absatz_Biermisch_hl|Versteuert_hl|Steuerfrei_hl|Export_EU_hl|
|Export_Drittland_hl|Haustrunk_hl|Braustaetten_Anzahl|Verbrauch_Bier_hl|Verbrauch
_pro_Kopf_l|
+----+-----+-----+-----+-----+-----+
+-----+-----+-----+-----+-----+-----+
-----+
|1994| 1.15660265E8|      NaN| 1.07356274E8| 8303991.0| 4289903.0|
|      3672039.0| 342048.0|      1299.0| 1.08006115E8|
132.7|
|1995| 1.1525206E8|      NaN| 1.06345563E8| 8906496.0| 4696448.0|
|      3882945.0| 327102.0|      1282.0| 1.07091697E8|
131.1|
|1996| 1.12806908E8|      NaN| 1.03506368E8| 9300540.0| 5128537.0|
|      3870892.0| 301110.0|      1276.0| 1.04290568E8|
127.3|
|1997| 1.12669814E8|      NaN| 1.03146796E8| 9523018.0| 5832665.0|
|      3404024.0| 286328.0|      1273.0| 1.03940973E8|
126.7|
|1998| 1.09384583E8|      NaN| 1.00180257E8| 9204326.0| 6132418.0|
|      2797534.0| 274374.0|      1285.0| 1.00908193E8|
123.0|
+----+-----+-----+-----+-----+-----+
+-----+-----+-----+-----+-----+-----+
-----+
only showing top 5 rows

```

Filter Data

Performe a filter operation on a column

```

In [11]: # Filtern nach Daten ab dem Jahr 2010
filtered_df = df.filter(df["Jahr"] >= 2010)
filtered_df.show(5)

```

```

+----+-----+-----+-----+-----+-----+
+-----+-----+-----+-----+-----+-----+
-----+
|Jahr|Absatz_Bier_hl|Absatz_Biermisch_hl|Versteuert_hl|Steuerfrei_hl|Export_EU_hl|
|Export_Drittland_hl|Haustrunk_hl|Braustaetten_Anzahl|Verbrauch_Bier_hl|Verbrauch
_pro_Kopf_1|
+----+-----+-----+-----+-----+-----+
+-----+-----+-----+-----+-----+-----+
-----+
|2010| 9.8350734E7| 3967715.0| 8.3431001E7| 1.4919733E7| 1.1028513E7|
| 3725289.0| 165930.0| 1333.0| 8.361995E7|
102.3|
|2011| 9.8293273E7| 3836189.0| 8.275982E7| 1.5533453E7| 1.1248396E7|
| 4126977.0| 158081.0| 1347.0| 8.3015185E7|
103.5|
|2012| 9.6531972E7| 4315453.0| 8.102393E7| 1.5508042E7| 1.1033069E7|
| 4324032.0| 150942.0| 1341.0| 8.1245431E7|
101.0|
|2013| 9.4645879E7| 4236569.0| 7.9717489E7| 1.492839E7| 1.0079292E7|
| 4701989.0| 147110.0| 1352.0| 7.9930195E7|
99.1|
|2014| 9.5666724E7| 4147606.0| 8.0065926E7| 1.5600797E7| 1.0135661E7|
| 5312992.0| 152144.0| 1359.0| 8.0280922E7|
99.1|
+----+-----+-----+-----+-----+-----+
+-----+-----+-----+-----+-----+-----+
-----+
only showing top 5 rows

```

Group By and Aggregate

Performe a group by and aggregat operation

```

In [12]: from pyspark.sql import functions as F

agg_df = filtered_df.groupBy().agg(F.avg("Verbrauch_pro_Kopf_1").alias("Avg_Verb
agg_df.show()

```

```

+-----+
|Avg_Verbrauch_Ab_2010|
+-----+
| 92.35625|
+-----+

```

Performe queries with Spark SQL

```

In [13]: # Temporäre SQL-Ansicht registrieren
df.createOrReplaceTempView("beer_table")

# SQL-Abfrage ausführen: Top 5 Jahre mit dem höchsten Bierabsatz (Absatz_Bier_hl
sql_results = spark.sql("""
    SELECT Jahr, Absatz_Bier_hl, Braustaetten_Anzahl
    FROM beer_table
    ORDER BY Absatz_Bier_hl DESC
    LIMIT 5

```

```
""")

sql_results.show()
```

```
+-----+-----+-----+
|Jahr|Absatz_Bier_hl|Braustaetten_Anzahl|
+-----+-----+-----+
|1994| 1.15660265E8|          1299.0|
|1995| 1.1525206E8|          1282.0|
|1996| 1.12806908E8|          1276.0|
|1997| 1.12669814E8|          1273.0|
|1999| 1.10148097E8|          1281.0|
+-----+-----+-----+
```

Save DataFrame to Parquet

```
In [14]: df.write.mode("overwrite").parquet("cleaned_beer_data.parquet")
```

```
In [15]: # import os
# import shutil
# import pandas as pd
#
# if os.path.exists("cleaned_beer_data.parquet"):
#     shutil.rmtree("cleaned_beer_data.parquet")
#
# print("Starte Speichervorgang...")
#
# os.makedirs("cleaned_beer_data.parquet", exist_ok=True)
#
# pdf_local.to_parquet("cleaned_beer_data.parquet/part-00000.parquet", index=False)
#
# with open("cleaned_beer_data.parquet/_SUCCESS", "w") as f:
#     pass
#
# print("DataFrame erfolgreich als Parquet gespeichert!")
```

```
In [16]: import pandas as pd

test_parquet = spark.read.schema(df.schema).parquet("cleaned_beer_data.parquet")
```

```
In [17]: spark.stop()
```